



Draw It or Lose It
CS 230 Project Software Design Document
Version 1.2

Table of Contents

CS 230 Project Software Design Template	1
Table of Contents	2
Document Revision History	2
Executive Summary	3
Requirements	3
Design Constraints	3
System Architecture View	4
Domain Model	4
Evaluation	5
Recommendations	8

Document Revision History

Version	Date	Author	Comments
1.0	05/24/2026	Rivera, Francella	Executive Summary, Design Constraints and Domain Model sections.
1.1	06/06/2026	Rivera, Francella	Evaluation and References.
1.2	06/19/2026	Rivera, Francella	Recommendations and References.

Executive Summary

Draw It or Lose It is a guessing game played in teams. It's played in four one-minute rounds. Each round random images are drawn by the system as clues, if the puzzle is not guessed within the minute the rest of teams are given an extra 15 seconds to make a guess.

Right now, this is a mobile-only game designed for one specific platform: Android. The goal is to turn this into a web-based application and make it available on all platforms.

This document contains the proposed architecture and design decisions to make it work on a web-based distributed environment.

The main decision behind the system design is to use Singleton pattern to make sure only one GameService object exists at a time, which matches the "only one instance of the game in memory at any given time" rule stated by the client. This architecture is meant to be scalable and able to adapt to any platform.

Requirements

Main requirements stated by the client are:

- A game will have the ability to have one or more teams involved.
- Each team will have multiple players assigned to it.
- Game and team names must be unique to allow users to check whether a name is in use when choosing a team name.
- Only one instance of the game can exist in memory at any given time. This can be accomplished by creating unique identifiers for each instance of a game, team, or player.

Design Constraints

Turning a one-platform application into a web-based system comes with new constraints and decisions to be made for it to work properly.

In this case (web application), one of the main concerns will be the connection. On a web application each player connects via browser to a single server (where the application lives). This has an added complexity so instead of a single player connected with its phone, we have multiple requests, users, and connections at the same time. This is why it's important to keep everything separate (by using unique IDs) so that the application can tell apart between games, players and teams.

Along with this comes one of the most important things to keep in mind: having only one instance of the game. And this is why the Singleton pattern matters that much since it will allow us to have one and only

one instance of the GameService. If by accident, we had multiple instances of a game then the players won't be connected to each other through a game but will be basically playing independently of each other.

Since this is a web-based system, the same application needs to work smoothly across different browsers, and platforms. This is an important design constraint because it needs to be built with a technology that supports this requirement, allowing for cross-browser and cross-platform compatibility of all features so any player can access it and join the game.

System Architecture View

Please note: There is nothing required here for these projects, but this section serves as a reminder that describing the system and subsystem architecture present in the application, including physical components or tiers, may be required for other projects. A logical topology of the communication and storage aspects is also necessary to understand the overall architecture and should be provided.

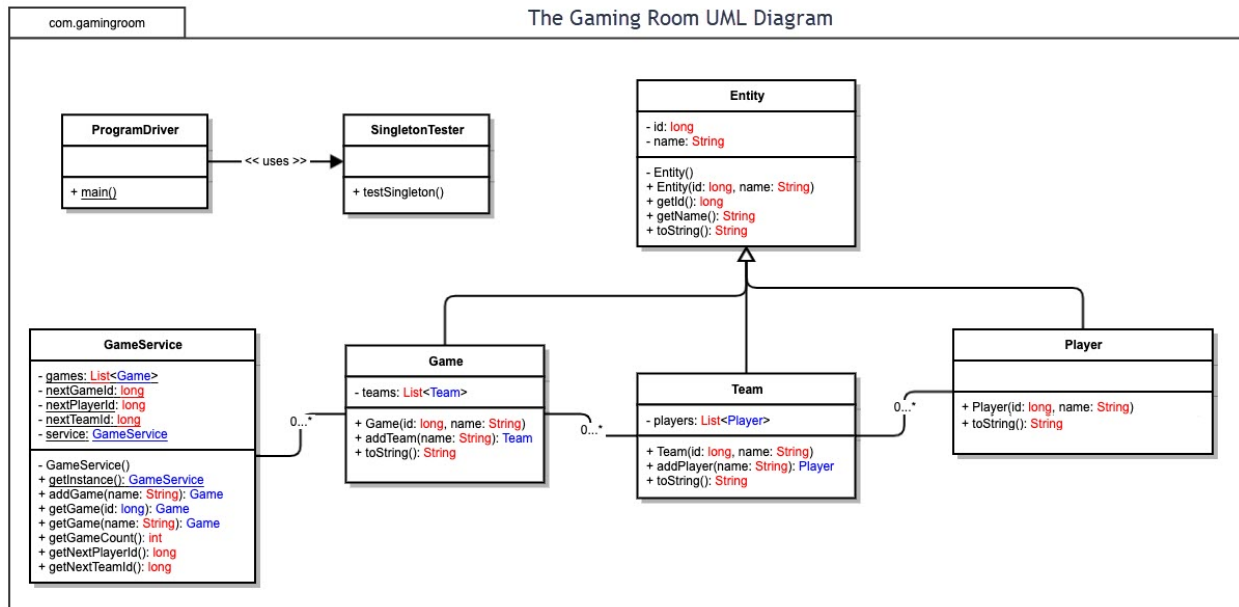
Domain Model

The UML Diagram shown below shows the design architecture for the Draw It or Lose It game. It consists of seven different classes each represented by a box on the diagram and how each connects to another. Each of the classes is divided in three different sections: the title (class name), the attributes (properties within the class), and the methods (actions we can access within the class).

The Entity class basically contains all methods and properties that are common for the three main classes (Game, Team, Player). All those three will inherit everything defined on Entity class, plus their own properties or methods. This is how we handle the unique names and ids for each.

As shown on the diagram we have different types of relationships between classes. In this case we see the 0..* relation between GameService and Game, which means there can be multiple games within one service, or said differently, a GameService owns a list of games. Same for the others, a Game has a list of Teams, and a Team has a list of Players.

GameService will be like the orchestrator or main class. It will handle everything and that's why the Singleton pattern is implemented here: so, we have a single instance on the application. Since the constructor is private (marked with "-") a new instance cannot be created from outside of the class, and the getInstance method will always returns the same instance of the class so it's not duplicated.



Evaluation

Using your experience to evaluate the characteristics, advantages, and weaknesses of each operating platform (Linux, Mac, and Windows) as well as mobile devices, consider the requirements outlined below and articulate your findings for each. As you complete the table, keep in mind your client's requirements and look at the situation holistically, as it all has to work together.

In each cell, remove the bracketed prompt and write your own paragraph response covering the indicated information.

See below for references used for this evaluation.

Development Requirements	Mac	Linux	Windows	Mobile Devices
Server Side	MacOS is based on Unix which makes it a good option for hosting since multiple tools like Apache, nginx and others are available. However back in 2022, Apple discontinued the macOS server and some hardware options making this a weak and not commonly used option for sever side hosting. Even though the OS is free (Unix based) and no extra licensing is required, Apple products tend to be pricey compared to others.	Linux is the industry standard for server hosting. Same as mac, it is Unix-based and most distributions are open source and free with no extra cost required for usage or license. It's a lightweight, performant and easy to scale option which makes it the best option for application hosting overall.	A Windows server is the go-to when working with an application that highly relies on Microsoft infrastructure, for example C# or .NET applications. However, that doesn't mean is the only case where it can be used. It's also a great option for application hosting but it's more propense to crashing, failures or security vulnerabilities and has a higher cost than Linux.	Mobile devices are not a good option for web application hosting due to multiple reasons, the main ones being connectivity, stability and memory. A web-based application like this game requires a stable and good connection because there will be multiple players connected at the same time. Mobile devices cannot offer that, also they can run out of battery and might not be as performant as other servers due to CPU and memory limitations which are difficult to scale.

Client Side	Since this is a web-based application the same HTML, CSS, and JS code runs on any macOS browser. The only mac-specific case to consider is Safari. Since this browser is only accessible on Mac devices it could add some cost if no devices are available for testing. It's important to do some extra testing on this browser because Safari uses WebKit engine which can sometimes behave differently than others like Chrome.	Linux is not that commonly used as OS on computers like others. However, even if users would access the application with this, they'll most likely use either Chrome or Firefox which completely support the same code as mentioned before. Linux does not add any cost on licensing or differentiated testing or access.	Windows has a similar case as MacOS. It supports the same common browsers as Chrome and Firefox. It also has an extra scenario which is Microsoft Edge, however this browser does not require additional support since it uses the same engine as Chrome.	All mobile devices like phones or tablets, normally support the exact same scenarios already mentioned. However, this case does require additional effort because the website needs to be responsive. This means extra development work and testing to make sure it adapts correctly to multiple devices and screen dimensions.
Development Tools	<u>Most used languages:</u> HTML, CSS, JS/TS and any backend language. <u>Tools:</u> Since macOS is Unix-based it uses zsh terminal and supports package managers like homebrew to install multiple tools. <u>IDEs:</u> Any, most used are VSCode, JetBrains (like IntelliJ).	<u>Most used languages:</u> HTML, CSS, JS/TS and any backend language. <u>Tools:</u> Uses terminal, Linux distributions like Ubuntu, Debian), and package managers such as apt and yum. <u>IDEs:</u> Any, most used are VSCode, JetBrains (like IntelliJ), Vim.	<u>Most used languages:</u> HTML, CSS, JS/TS and C#, .NET for backend (most supported ones). <u>Tools:</u> WSL (Windows Subsystem for Linux) to run a full Linux env. <u>IDEs:</u> Most used are VSCode, and Visual Studio.	Same web code can run on any mobile device browser. They are mostly used for testing purposes either on physical devices or emulators. However, development perse is not commonly done on mobile devices.

Recommendations

Analyze the characteristics of and techniques specific to various systems architectures and make a recommendation to The Gaming Room. Specifically, address the following:

1. **Operating Platform:** The recommended for hosting is Linux. As mentioned on the evaluation section before, this operating system is the industry standard due to its characteristics such as open source, lightweight, easy to scale, and others. Also, all three major cloud providers offer Linux as default option which gives free choice on the hosting provider.
2. **Operating Systems Architecture:** Linux uses a monolithic kernel with loadable modules, providing flexibility while maintaining high performance. It separates all operations into kernel space, and user space. Kernel space is where it interacts with the hardware and system resources, while user space is where applications such as this game will run. [10]
3. **Storage Management:** Since we have an enormous image library for the game, the recommendation is to rely on a cloud storage provider such as AWS S3. Linux offers a built in ext4 file system which is great, however will not scale as expected if this image set keeps growing. For the rest of the game data, the recommendation is to host a DB on the Linux server such as PostgreSQL.
4. **Memory Management:** Linux uses virtual memory and demand paging to manage memory efficiently. Virtual memory gives each process its own memory space, and on the other hand, demand paging loads data on RAM only when needed. Another important part is Linux's cache which speeds up repeated requests such as images by keeping them in memory. By leveraging all these Linux's features, we can ensure stability, performance and scalability on the server. [9]
5. **Distributed Systems and Networks:** Since users will be connecting to the game via multiple devices and different locations this is a distributed system. The ideal scenario would be that the backend (core game logic) functionality is provided via an HTTPS API which will guarantee both security and accessibility through any client (any browser, device). Since this game requires real-time updates, we would need to implement this via WebSocket ^[8] or any other alternative that offers real time communication sessions between client and server.

6. **Security:** Since this will be accessible through the web, it is a must that the team sets up traffic encryption via HTTPS which makes it a secure website and keeps data from being read through the network. Another important thing is that if we want users to access via password, those will also need to be encrypted instead of just stored as plain text. Linux itself provides good security which could be improved with extra built-in tools that it offers.

References

- [1] Rivera, J. (2022, September 19). *Windows vs macOS vs Linux for Web Design & Development*. PixemWeb. Retrieved June 6, 2026, from <https://www.pixemweb.com/blog/windows-vs-macos-vs-linux-for-web-design-development/>
- [2] Whatley, S. (2024, March 27). *Comparing Mac and Windows Servers for Web Hosting*. Mamboserver. Retrieved June 6, 2026, from <https://www.mamboserver.com/blog/comparing-mac-and-windows-servers-for-web-hosting/>
- [3] [VS Tech]. (2026, February 24). *Windows vs macOS vs Linux for Developers – Which Operating System Is Best for Programming in 2026?* [Video]. YouTube. <https://m.youtube.com/watch?v=hwlu1jr0xUo&t=43s&pp=2AErkAIB>
- [4] Unknown (2026, March 11). *Chrome (Blink) vs Safari (WebKit): A Comparison of Browser Engines*. ToDetect. Retrieved June 6, 2026, from <https://www.todetect.net/article/browser-checker/web-compatibility/>
- [5] [VS Tech]. (2025, November 02) *Best Operating System for Programming: Windows, Linux, or macOS?* [Video]. YouTube. <https://m.youtube.com/watch?v=yVxt6T-c0M&pp=0gcJCSgLAYcqIYzv>
- [6] Hamada, P. (2025, February 04). *The End of Apple's macOS Server*. Miradores. Retrieved June 6, 2026, from <https://www.miradore.com/blog/the-end-of-apple-macos-server/>
- [7] Islam, N. (2024, May 27). *Choosing the Right OS: Mac, Windows, or Linux — A Programmer's Guide!* Medium. Retrieved June 6, 2026, from <https://medium.com/@nomannayeem/choosing-the-right-os-mac-windows-or-linux-a-programmers-guide-2d128c40fc73>
- [8] MDN Contributors. (2026). *WebSocket API (WebSockets)*. MDN. Retrieved June 19, 2026, from https://developer.mozilla.org/en-US/docs/Web/API/WebSockets_API
- [9] Lambdafunc. (2024, Nov 15). *Linux — Memory Management*. Medium. Retrieved June 19, 2026, from <https://lambdafunc.medium.com/linux-memory-management-2c13b5e386f1>
- [10] Basharat, M. (2026, Mar 11). *Linux Kernel Architecture Overview*. Medium. Retrieved June 19, 2026, from <https://medium.com/@majidbasharat21/linux-kernel-architecture-overview-9566f9617049>